

# SOLEMNE 2 – Desarrollo de Interfaz Frontend

Universidad San Sebastián

Aplicaciones y Tecnologías de la Web

Sección: NRC-2467

Integrantes: Gabriel Cárdenas, Matías Gómez, Ignacio Matamoras, Santiago Núñez

Profesor: René Galarce

# 1. Contexto del proyecto

## 1.1 Descripción del sistema VCM:

- **Propósito:** Desarrollar una interfaz frontend ágil y eficiente para la gestión de tickets de mantenimiento en infraestructura de tiendas (iluminación, climatización, escaleras, etc.). El sistema busca permitir el registro inmediato de incidencias, asegurando que no se interrumpa la operatividad del local y facilitando la priorización de fallas críticas.
- **Usuarios objetivo:** El sistema está diseñado para colaboradores de tienda y personal de mantenimiento o jefaturas.
- **Problema que resuelve:** Actualmente, los reportes de mantenimiento se realizan de forma tardía, carecen de contexto técnico y no cuentan con criterios claros de prioridad. Esto genera retrasos en la resolución de fallas que afectan la experiencia del cliente y la seguridad operativa. La plataforma centraliza la información y asigna prioridades claras (niveles de riesgo alto/bajo).

## 1.2 Alcance de la interfaz desarrollada:

El sistema implementa un flujo de usuario enfocado en la agilidad y eficiencia del reporte de fallas.

Paginas:

- Homepage:** Página principal que presenta un dashboard con estadísticas generales del sistema, permitiendo la visualización del estado de los tickets.
- Tickets:** Pagina con un listado completo de todos los tickets del sistema.
- Contacto:** Pagina dedicada a la comunicación con el soporte, donde se pide el nombre completo de la persona, el email, el asunto y finalmente una casilla donde explicarse con lo que desea comunicar.
- Accede a tu cuenta:** Acceso para diferentes roles (usuario/admin), donde al acceder como usuario puedes elegir entre distintas opciones, tales como, registro de comunicaciones, ayuda, volver y nuevo ticket, este último te abre un formulario el cual debes completar con tu nombre de usuario, tipo de falla, zona de la falla, descripción y evidencia donde podrás adjuntar un archivo donde se muestre la falla.
- Regístrate:** Pagina de registro para usuarios nuevos.

### 1.3 Justificación de diseño:

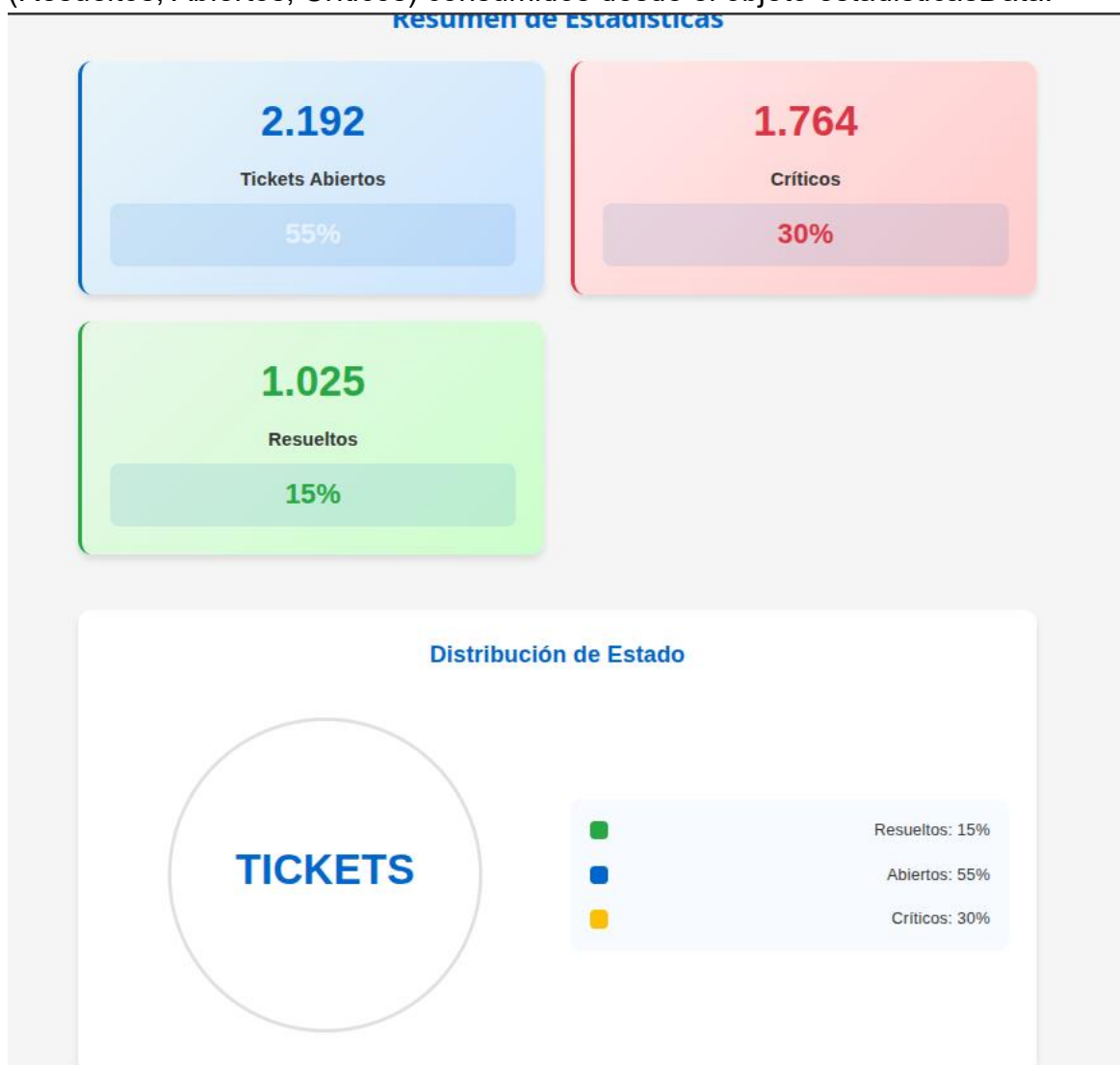
- Identidad y Confianza Corporativa: Se ha adoptado una paleta cromática dominada por tonos azules (característicos de la identidad visual de Cencosud) para transmitir profesionalismo, estabilidad y confianza en el entorno de gestión de infraestructura. Este uso del color permite que los usuarios (colaboradores y personal de mantenimiento) perciban la plataforma como una herramienta oficial y fiable, esencial para la operación en tienda.
- Codificación de estados (Semáforo): Para categorizar la criticidad de los tickets. Esta decisión permite que el personal tome decisiones rápidas basadas en la urgencia y el riesgo, cumpliendo con el objetivo de no interrumpir la operatividad.
- Jerarquía Visual y Experiencia de Usuario (UX): La disposición de los elementos en el dashboard y formularios prioriza la legibilidad y la rapidez. La estructura de tarjetas para los tickets facilita el escaneo visual de información, garantizando que el usuario pueda registrar o supervisar una falla en segundos.
- Consistencia Visual: La interfaz asegura que, independientemente de la página en la que se encuentre el usuario, la experiencia sea predecible. La adopción de espacios, tipografías y botones alineados con la identidad de marca de la organización asegura que la curva de aprendizaje del sistema sea mínima, facilitando su adopción rápida en las distintas sucursales.

## 2. Requerimientos del Sistema

### 2.1 Requerimientos Funcionales

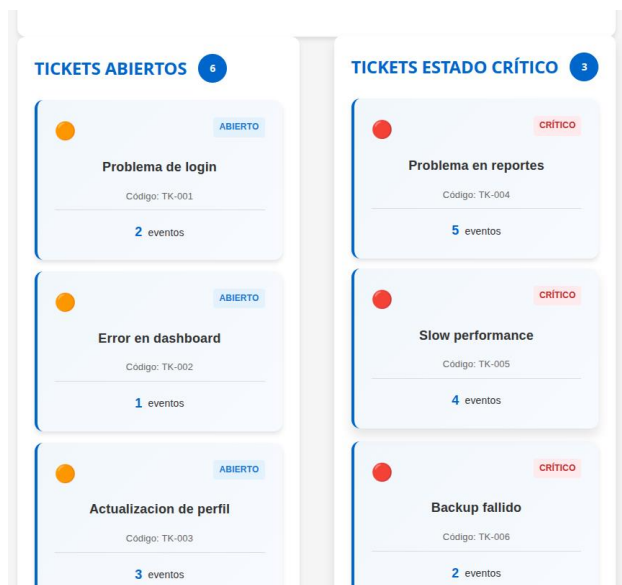
RF-1:

- La Homepage debe mostrar al menos un resumen estadístico con datos estáticos.
- Página donde se implementa: Homepage (./src/pages/Home.jsx)
- Evidencia de cumplimiento: Se implementó una sección “Distribución de Estado” en el layout principal que renderiza datos estadísticos duros (Resueltos, Abiertos, Críticos) consumidos desde el objeto estadísticasData.



RF-2:

- Descripción: Flujo de Gestión (adaptado a Tickets): Listado con al menos 6 ítems estáticos renderizados mediante un componente reutilizable, vista de detalle (navegación interna) y formulario de registro con validación visual.
- Página donde se implementa: Homepage (/), Tickets (/tickets/:id), Contacto (/contacto).
- Evidencia de cumplimiento: En la Homepage se iteran los elementos utilizando el componente <TicketCard />. Se configuraron las rutas dinámicas en App.jsx para permitir la navegación al detalle de cada ítem, y se utilizaron formularios controlados con validación en las vistas correspondientes.



### RF-3:

- Descripción: La Página de Contacto (o acciones principales) debe mostrar confirmación o error al enviar el formulario mediante el Componente <Mensaje/>.
- Página donde se implementa: Contacto (/contacto).
- Evidencia de cumplimiento: Creación del componente <Mensaje tipo="exito" texto="..." />. Su renderizado está condicionado por un estado booleano y de texto gestionado mediante el hook useState, montándose dinámicamente en el DOM al simular cargas o envíos.



Nombre:

Correo:  ! Completa este campo

Formulario de Contacto

Nombre:

Correo:

Asunto:

Mensaje:

¡Formulario enviado con éxito!

Aceptar

RF-4:

- Descripción: La navegación entre páginas debe implementarse con React Router y estar gestionada por el Componente Menú.
- Página donde se implementa: App.jsx y MenuNav.jsx
- Evidencia de cumplimiento: Uso de <BrowserRouter>, <Routes> y <Route> para definir el enrutamiento general. El componente <MenuNav /> recibe un array de rutas por props y utiliza la etiqueta <Link> de react-router-dom para transiciones sin recargar el navegador.



```
return (  
  <Router>  
    <div className="app-container">  
      /* CSS Grid layout principal incluye el Header */  
      <header className="app-header">  
          
        <MenuNav links={enlacesMenu} />  
      </header>  
  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/tickets" element={<div>Página de Listado de Tickets</div>} />  
        <Route path="/tickets/:id" element={<div>Detalle del Ticket</div>} />  
        <Route path="/contacto" element={<div>Página de Contacto</div>} />  
        /* Rutas extra para cumplir los enlaces del header */  
        <Route path="/login" element={<div>Login</div>} />  
        <Route path="/registro" element={<div>Registro</div>} />  
      </Routes>  
  
      <footer className="app-footer">  
        <p>&copy; 2026 Sistema de Tickets</p>  
      </footer>  
    </div>  
  </Router>  

```

#### RF-5:

- Descripción: Los datos deben almacenarse en un archivo de datos estáticos y ser consumidos mediante useState / props.
- Página donde se implementa: Directorio de datos (src/data/tickets.js) y Home.jsx.
- Evidencia de cumplimiento: Creación de un array de objetos exportable con la información estática. Estos datos se importan en el componente principal y se inyectan en el estado local mediante const [tickets, setTickets] = useState(ticketsData), para luego pasarse como props a las tarjetas.

```
import { useState, useEffect } from 'react';
import { ticketsData, estadisticasData } from '../data/tickets';
import { TicketCard } from '../components/TicketCard';
import './Home.scss';

export const Home = () => {
  const [tickets, setTickets] = useState([]);
  const [filteredTickets, setFilteredTickets] = useState([]);
  const [stats, setStats] = useState(estadisticasData);
}
```

< > ... > solemne2-tecnologias-y-aplicaciones-web > solemne2-apps-web > src > data

| Nombre ▾                                                                                     | Modificado     | Tamaño |
|----------------------------------------------------------------------------------------------|----------------|--------|
|  tickets.js | Hoy, 3:56 p.m. | 1.4 KB |

## 2.2 Requerimientos No Funcionales

### RNF-1:

- Descripción: Uso correcto de <label> con htmlFor, atributo alt en imágenes, orden lógico de tabulación, contraste de colores (mínimo 4.5:1) y navegación por teclado.
- Evidencia de cumplimiento:
  - El buscador de tickets utiliza un <label> vinculado al id del input.
  - Todas las etiquetas <img> (logo y avatares) incluyen el atributo alt descriptivo.
  - La paleta de colores en SASS garantiza contraste (ej. texto oscuro o blanco puro sobre el azul corporativo #005A9C).
  - Se utilizaron enlaces y botones nativos que respetan el índice de tabulación natural del DOM, permitiendo recorrer el menú y las tarjetas solo con la tecla Tab.

### RNF-2:

- Descripción: El proyecto debe utilizar obligatoriamente las siguientes etiquetas: <header>, <nav>, <main>, <section>, <article>, <aside>, <footer>, <h1>, <h2>, <h3>, <p>, <ul>.
- Evidencia de cumplimiento: El maquetado de la aplicación respeta los estándares de HTML5. En la estructura global se utiliza <header> y <footer>. El componente de navegación utiliza <nav> y listas desordenadas <ul>. El



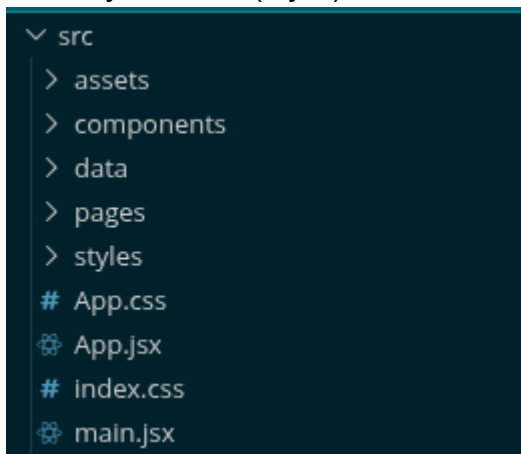
contenido de la vista principal se divide lógicamente utilizando `<main>`, `<aside>` (para la barra lateral), `<section>` (para los grupos de tarjetas), y jerarquía de encabezados (`<h2>`, `<h3>`) y párrafos (`<p>`) dentro de las tarjetas (que actúan como `<article>`).

#### RF-3:

- Descripción: Adaptación de layouts según puntos de ruptura (Escritorio > 1024px, Tablet 768-1023px, Móvil < 768px).
- Evidencia de cumplimiento: Uso riguroso de Media Queries en los archivos .scss. Se implementó CSS Grid en el layout base: en escritorio mantiene un formato multi-columna; en tablet ajusta proporciones; y en móvil (max-width: 767px) el `<aside>` se desplaza debajo del `<main>`, el grid de tarjetas pasa a una sola columna y el menú de navegación cambia a disposición vertical con Flexbox.

#### RF-4:

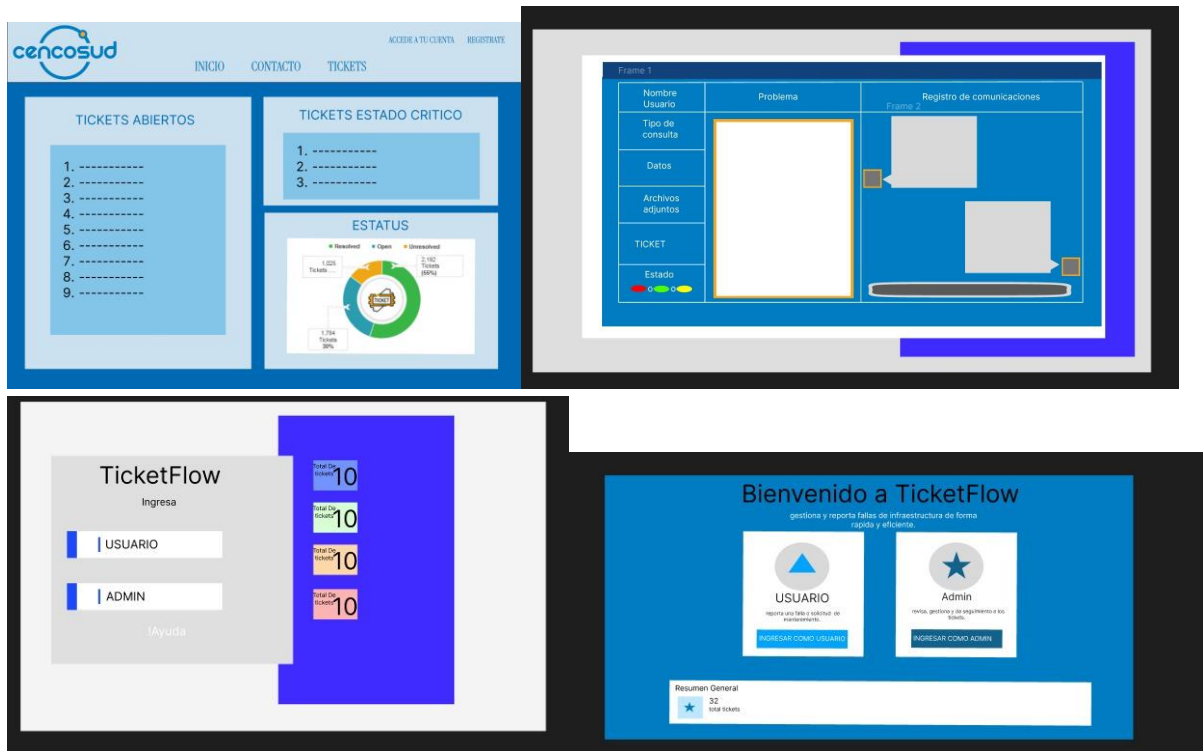
- Descripción: Componentes en carpetas separadas, estilos modulares en SASS/SCSS y consola sin errores.
- Evidencia de cumplimiento: La arquitectura del proyecto separa la lógica en /components, las vistas completas en /pages, la información en /data y los estilos en /styles. Se emplearon parciales de SASS (`_variables.scss`, `_mixins.scss`) importados en un archivo principal. La navegación por la app no arroja errores (rojos) de React en la consola del navegador.



### 3. Desarrollo de la interfaz (UI/UX)

#### 3.1 Diseño de la Interfaz

Wireframes y prototipos del sistema:



La paleta de colores elegida para este proyecto varió mucho a lo largo del desarrollo de este, al final nos decidimos por una paleta de colores diferente a la de la página principal que llevaría a la nuestra para demostrar un cambio notable entre las funciones de la dos páginas, como la nuestra se centra en atender las necesidades de los empleados de la empresa se necesita demostrar esa diferencia entre lo que ven los clientes (ajenos a la empresa) y lo que ven los empleados involucrados con la misma

Si el usuario desea las estadísticas generales del sistema de tickets, lo primero que querría ver el usuario que necesita las estadísticas seria la cantidad de tickets abiertos, la cantidad de tickets críticos y la posiblemente la cantidad de resueltos para comprobar la productividad del sistema, para lograr esto aumentamos el tamaño de la métrica cuantitativa y les asignamos colores con el fin de demostrar a que grupo pertenece dicha métrica, estos van a ser los únicos colores “externos” a la paleta principal logrando que resalten aún más.

En el caso del resto del sistema usamos la misma paleta de colores original para mantener uniformidad y armonía visual.

## 3.2 Desarrollo HTML Semántico

El proyecto implementa una arquitectura basada en **HTML5 Semántico** y prácticas de accesibilidad, asegurando que el contenido sea comprensible tanto para usuarios como para herramientas de asistencia (lectores de pantalla).

### Estructura Global y Contenedores Semánticos

El maquetado base de la aplicación reemplaza el uso excesivo de contenedores genéricos (<div>) por elementos estructurales de HTML5 que definen claramente las zonas de la interfaz para el navegador.

### Evidencia de cumplimiento:

**-header:** Encapsula de forma semántica el área superior institucional de Cencosud, conteniendo el logo y el título principal.

**-nav:** Define el bloque de navegación global del sistema (.menu-nav), albergando las listas desordenadas (<ul>) y los elementos de enlace.

**-main:** Envuelve el contenido troncal y dinámico de cada vista (.home, .contacto-page, etc.), indicando al navegador el núcleo de la información.

**-footer:**

```
<!DOCTYPE html>
<html lang="en">
  <head>
  </head>
  <body>
    <div id="root">
      <div class="app-container">
        <header class="app-header">
        </header>
        <div class="home">
        </div>
        <footer class="app-footer">
        </footer>
      </div>
    </div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

### Jerarquía de Encabezados y Contenidos

Uso riguroso de etiquetas de encabezado (h1, h2, h3) y párrafos (<p>) para establecer un orden jerárquico visual y estructural correcto. Las tarjetas de información y tickets se aíslan semánticamente.

## Evidencia de cumplimiento:

- h1: Utilizado exclusivamente para el título principal de cada página (.contacto-page h1, .auth-page h1).
- h2 Implementado en los subtítulos de las secciones del dashboard (ej. .home\_\_section-title).
- article: Cada tarjeta de ticket (.ticket-card) se maqueta bajo esta etiqueta, indicando que es un elemento independiente y autocontenido dentro del flujo.

```

1 import './TicketCard.scss';
2
3 export const TicketCard = ({ nombre, codigo, estado, cantidad, imagen = '■' }) => {
4   const estadoClass = estado.toLowerCase().replace(' ', '-');
5
6   return (
7     <div className="ticket-card" role="article">
8       <div className="ticket-card__header">
9         <span className="ticket-card__icon" aria-hidden="true">{imagen}</span>
10        <span className={`ticket-card__estado ticket-card__estado--${estadoClass}`}>
11          {estado}
12        </span>
13      </div>
14
15      <div className="ticket-card__content">
16        <h3 className="ticket-card__nombre">{nombre}</h3>
17        <p className="ticket-card__codigo">Código: {codigo}</p>
18
19        <div className="ticket-card__footer">
20          <span className="ticket-card__cantidad">
21            <strong>{cantidad}</strong> eventos
22          </span>
23        </div>
24      </div>
25    </div>
26  );
27 };
28
29 export default TicketCard;
30

```

## Formularios Accesibles y Vinculados

Maquetado de formularios técnicos garantizando la accesibilidad mediante el uso correcto de elementos nativos y vinculación estricta de campos.

## Evidencia de cumplimiento:

- htmlFor e id: Vinculación estricta en cada campo para permitir que los lectores de pantalla identifiquen el propósito del control y que el usuario pueda hacer foco clickeando la etiqueta.
- Controles Nativos: Uso de elementos semánticos estándar como <select> con sus respectivas <option> para la selección de tipo de falla (Iluminación, Climatización, etc.) o zona (Cajas, Pasillo, Bodega, Entrada), y <textarea> para descripciones extensas.
- <button type="reset"> y type="submit": Definición explícita del comportamiento de los botones del formulario (.acciones), donde uno restablece los campos de forma nativa ("Limpiar") y el otro procesa el envío de la solicitud ("Crear ticket").

```
function Login() {
  <h3>Nuevo ticket</h3>
  <p>Completa el formulario para crear tu solicitud.</p>

  <div className="fila">
    <div>
      <label htmlFor="nombre">Nombre</label>
      <input id="nombre" type="text" placeholder="Nombre usuario" />
    </div>

    <div>
      <label htmlFor="tipo">Tipo de falla</label>
      <select id="tipo">
        <option>Selecciona el tipo de falla</option>
        <option>Iluminación</option>
        <option>Climatización</option>
        <option>Enchufes</option>
        <option>Puertas</option>
      </select>
    </div>
  </div>

  <label htmlFor="zona">Zona</label>
  <select id="zona">
    <option>Selecciona la zona o área</option>
    <option>Cajas</option>
    <option>Pasillo</option>
    <option>Bodega</option>
    <option>Entrada</option>
  </select>

  <label htmlFor="descripcion">Descripción del problema</label>
  <textarea
    id="descripcion"
    rows="4"
    placeholder="Describe detalladamente el problema"
  ></textarea>

  <label htmlFor="archivo">Adjuntar archivo</label>
  <input id="archivo" type="file" />

  <div className="acciones">
    <button type="reset">Limpiar</button>
    <button type="submit" className="crear">
      Crear ticket
    </button>
  </div>
</form>
}
```

### 3.3 Desarrollo CSS con Preprocesador SCSS

#### Arquitectura de archivos SCSS

El proyecto adopta una arquitectura modular, organizando los estilos en archivos independientes (*partials*) que separan las responsabilidades de la interfaz de usuario. Cada vista principal posee su propio archivo de estilos, los cuales importan configuraciones globales de diseño:

- **\_variables.scss:** Archivo base para la definición global de la paleta cromática, tipografías, espaciados y *breakpoints*.
- **\_mixins.scss:** Biblioteca de funciones y bloques de código reutilizables (Flexbox, Grid y efectos visuales).
- **Header.scss / Footer.scss:** Estilos específicos para la identidad institucional y el cierre de la aplicación.
- **TicketCard.scss:** Componente central que maneja la lógica visual y los estados de las tarjetas de soporte de Cencosud.

- **Mensaje.scss / Contacto.scss / Home.scss:** Estilos aplicados a las alertas de feedback, formularios de contacto y la estructura general del dashboard principal.

## Variables y Anidamiento (Nesting)

Se emplean variables para organizar aspectos visuales como los colores y las fuentes, logrando una mayor uniformidad en el diseño. Además, el anidamiento facilita la creación de estilos siguiendo el orden estructural del HTML.”

### Evidencia de cumplimiento:

```
.mensaje {
  @include flex-between;
  padding: $espaciado-md $espaciado-lg;
  border-radius: $border-radius-md;
  margin-bottom: $espaciado-md;
  gap: $espaciado-md;
  box-shadow: $sombra-md;
  animation: slideIn 0.3s ease-out;

  &__contenido {
    @include flex-between;
    gap: $espaciado-md;
    flex: 1;
    align-items: center;
  }
}
```

```
&--exito {
  background-color: #d4edda;
  color: #155724;
  border-left: 4px solid #28a745;

  .mensaje__icono {
    color: $color-exito;
  }
}

&--error {
  background-color: #f8d7da;
  color: #721c24;
  border-left: 4px solid $color-error;

  .mensaje__icono {
    color: $color-error;
  }
}
```

## Mixins

Con el propósito de evitar la repetición de código, se utilizaron mixins tanto estáticos como parametrizados para reutilizar configuraciones asociadas a Flexbox, Grid y diseños adaptables, facilitando así el mantenimiento del proyecto.

#### Evidencia de cumplimiento:

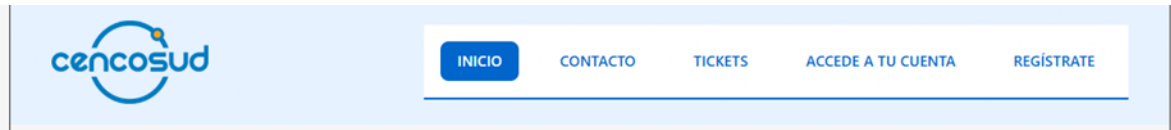
```
60 @mixin card {  
61   background-color: $color-blanco;  
62   border-radius: $border-radius-lg;  
63   padding: $espaciado-lg;  
64   box-shadow: $sombra-md;  
65   transition: all 0.3s ease;  
66 }
```

```
@mixin flex-between {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
}
```

#### Layout con Flexbox (Menú, tarjetas y formularios)

La estructura de menús, formularios y tarjetas se desarrolla mediante Flexbox, lo que facilita la correcta disposición de los componentes internos y garantiza un alineamiento uniforme entre los elementos.

### Evidencia de cumplimiento:



```
.header {  
  background: linear-gradient(135deg, $color-secundario 0%, #cce5ff 100%);  
  border-bottom: 3px solid $color-primario;  
  padding: $espaciado-lg $espaciado-md;  
  
  .container {  
    max-width: 1200px;  
    margin: 0 auto;  
    padding: 0 $espaciado-md;  
  }  
  
  &__content {  
    @include flex-between;  
    align-items: center;  
    gap: $espaciado-lg;  
  
    @include responsive-mobile {  
      flex-direction: column;  
      text-align: center;  
      gap: $espaciado-md;  
    }  
  }  
}
```



## Layout con CSS Grid (Layout Principal)

La distribución de bloques de información en el layout principal se realizó mediante CSS Grid, ya que permite organizar los elementos de forma más exacta dentro de la página.

### Evidencia de cumplimiento:

```
.home {
  background-color: $color-fondo;
  min-height: calc(100vh - 200px);
  padding: $espaciado-xxl $espaciado-md;
}

> .container { ...
}

  &_main {
    display: grid;
    grid-template-columns: 1fr;
    grid-template-rows: auto;
    gap: $espaciado-xxl;
  }

> &_section-title { ...
}

> &_estadisticas { ...
}

> &_tickets { ...
}

> @include responsive-tablet { ...
}

> @include responsive-mobile { ...
}

}

// Estadísticas Grid
.estadisticas-grid {
  @include grid-responsive(3, $espaciado-lg);
  margin-bottom: $espaciado-xxl;

  @include responsive-tablet {
    grid-template-columns: repeat(2, 1fr);
    gap: $espaciado-md;
  }

  @include responsive-mobile {
    grid-template-columns: 1fr;
    gap: $espaciado-md;
  }
}
```



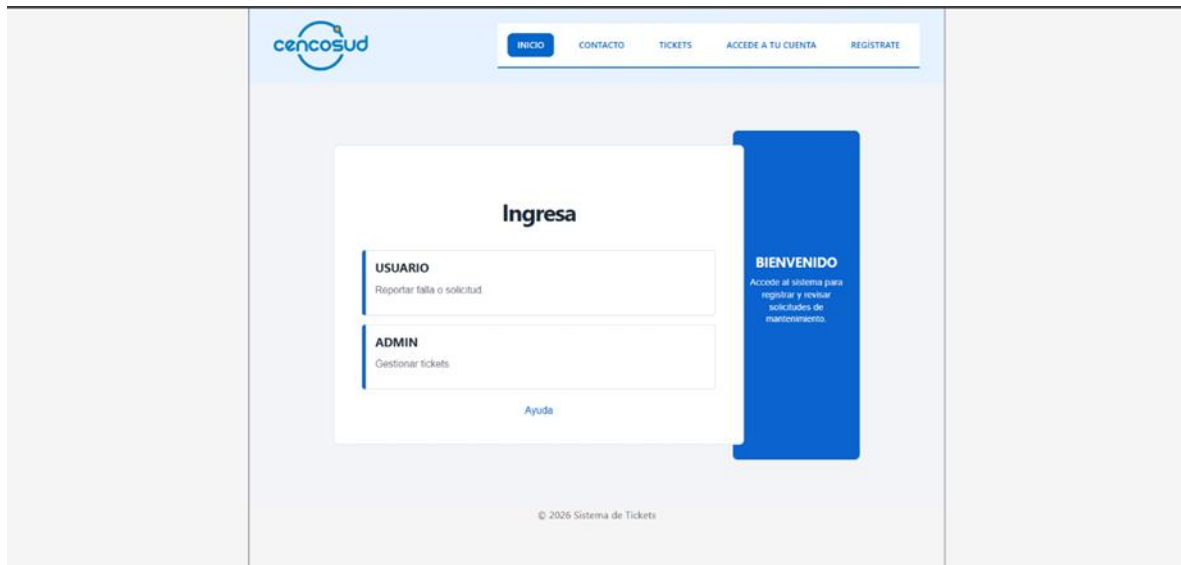
## Evidencia de Diseño Responsivo (Breakpoints)

Adaptación fluida de los layouts según los tres puntos de ruptura estandarizados en el proyecto.

### Evidencia de cumplimiento:

#### Escritorio / Desktop (Resoluciones superiores a 1024px)

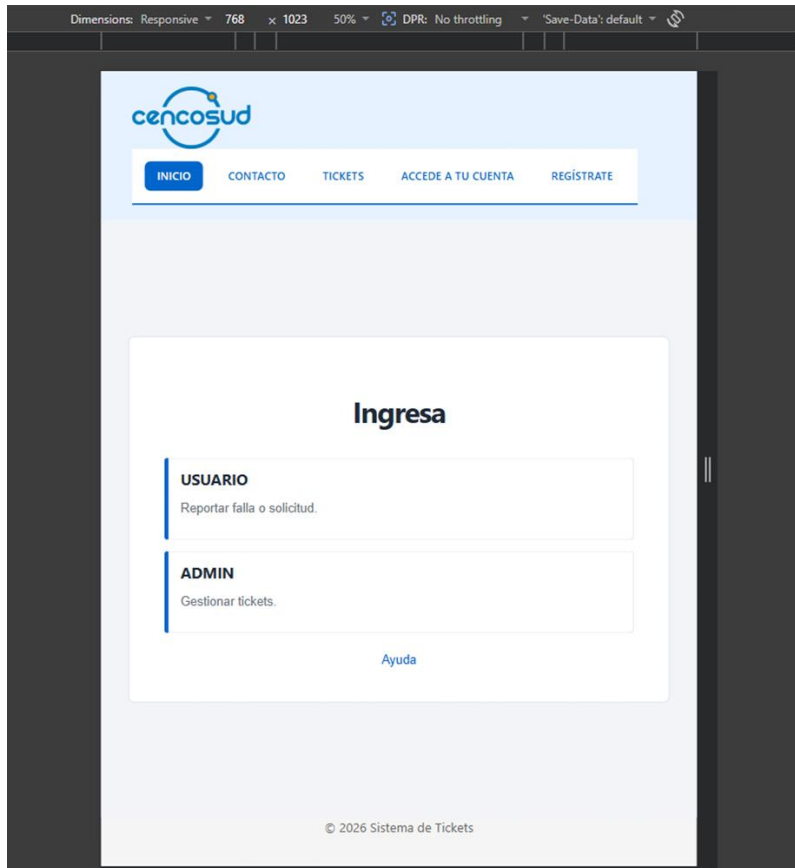
El dashboard se muestra en su máximo esplendor con las estadísticas distribuidas en 3 columnas y el contenedor de tickets en 2 columnas mediante Grid de forma paralela.



#### Tablet (Resoluciones entre 768px y 1023px)

El grid de estadísticas se reajusta dinámicamente a 2 columnas y el contenedor de tickets colapsa a una única columna para mantener la legibilidad de la información.

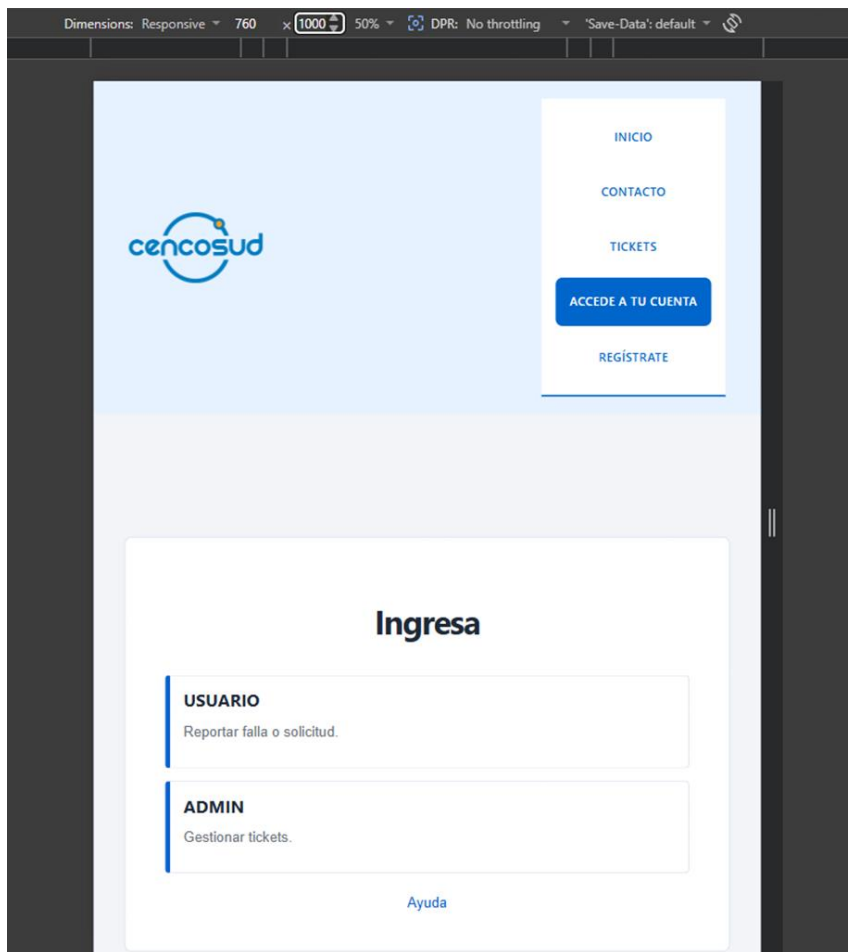
```
@include responsive-tablet {  
  padding: $espaciado-lg $espaciado-md;  
}
```



## Mobile / Smartphone (Resoluciones inferiores a 768px)

Toda la interfaz colapsa a una sola columna estricta (1fr). Los elementos de navegación del menú y del header cambian su propiedad flex-direction a column, ordenándose verticalmente para facilitar la usabilidad táctil.

```
41  
42 ✓ @include responsive-mobile {  
43   padding: $espaciado-md;  
44 }  
45 }
```



## 3.4 Desarrollo en React

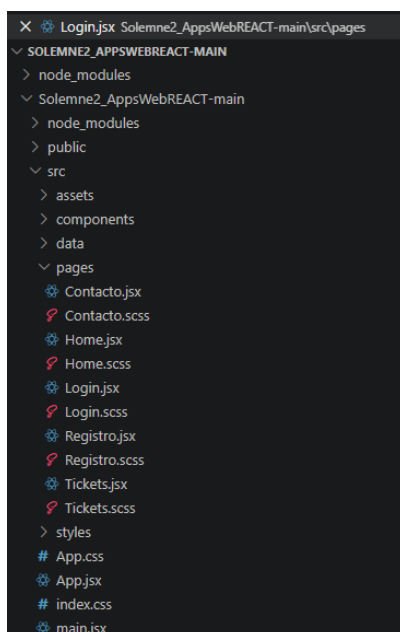
### Contexto y objetivo

El desarrollo en React tuvo como objetivo organizar la interfaz del sistema de tickets en componentes reutilizables, facilitando la separación entre estructura, datos, estilos y navegación. La aplicación fue construida con React y Vite, utilizando archivos .jsx para las vistas y componentes, y archivos .scss para los estilos visuales.

La lógica principal se centra en mostrar una página de inicio con accesos al usuario y administrador, además de un resumen general de tickets. Desde la vista de usuario se incorpora un formulario para crear una solicitud de mantenimiento, un registro de comunicaciones y un estado actual del ticket. Esta estructura permite representar el flujo básico de gestión de tickets solicitado en la solemne.

### Árbol de componentes de la aplicación

La aplicación se organizó separando páginas completas, componentes reutilizables y estilos. La estructura usada fue la siguiente:



En esta estructura, App.jsx funciona como el archivo que organiza las rutas principales de la aplicación. Las páginas se encuentran dentro de /pages, mientras que los elementos reutilizables se encuentran dentro de /components. Esta separación permite que el código sea más ordenado y fácil de mantener.

### Componentes reutilizables

React permite dividir la interfaz en partes más pequeñas llamadas componentes. En el proyecto se utilizaron componentes reutilizables para evitar repetir código y mantener una estructura más clara.

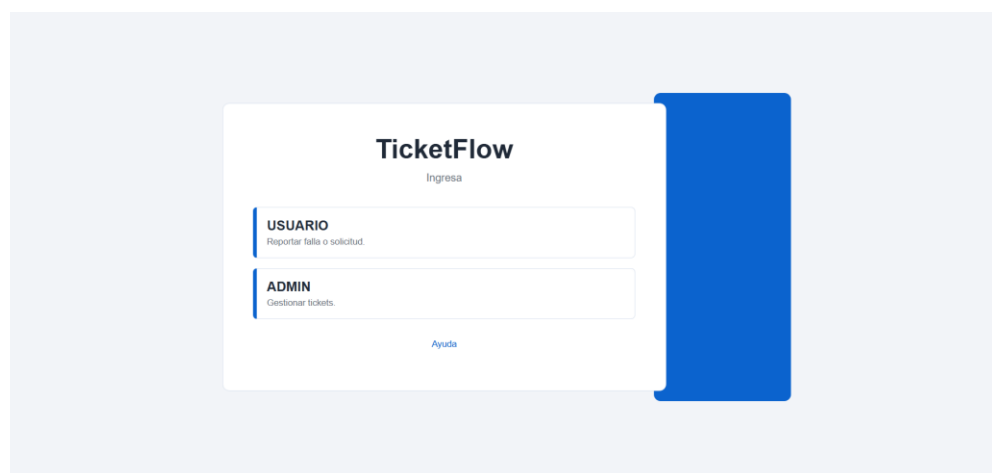
## Componente HomeCard

El componente HomeCard se utilizó para representar las opciones principales de acceso en la pantalla inicial. En este caso, se usó para mostrar las tarjetas Usuario y Admin, manteniendo la misma estructura visual y evitando repetir código.

Este componente recibe información mediante props. La prop titulo muestra el nombre principal de la tarjeta, descripcion entrega una explicación breve y onClick define la acción que se ejecuta cuando el usuario presiona la tarjeta.

```
Solemne2_AppsWebREACT-main > src > components > HomeCard.jsx > default
1  function HomeCard({ titulo, descripcion, onClick }) {
2    return (
3      <button className="home-card" onClick={onClick}>
4        <h2>{titulo}</h2>
5        <p>{descripcion}</p>
6      </button>
7    );
8  }
9
10 export default HomeCard;
```

En la pantalla de ingreso, el componente se reutiliza de la siguiente forma:



```
titulo="USUARIO"
descripcion="Reportar falla o solicitud."
onClick={() => {
  setVista("usuario");
  setSeccion("nuevo");
}}
/>

<HomeCard
  titulo="ADMIN"
  descripcion="Gestionar tickets."
  onClick={() => alert("Vista administrador pendiente")}
/>
```

La lógica principal está en el evento onClick. En la tarjeta de usuario, al hacer clic, se actualiza el estado de la vista y se muestra el panel correspondiente al formulario de tickets. Esto permite que la interacción ocurra dentro de la misma página, sin recargar el navegador.

### Componente StatCard

El componente StatCard se utilizó para representar tarjetas de resumen dentro de la interfaz. Su objetivo es mostrar información breve de manera ordenada, por ejemplo la cantidad total de tickets, tickets pendientes, críticos o resueltos. Aunque en la versión visual actual el panel azul puede quedar vacío por decisión de diseño, este componente sigue siendo importante para explicar la lógica reusable del proyecto.

El componente recibe tres props principales: titulo, numero y tipo. La prop titulo indica el nombre del dato que se muestra, numero representa el valor asociado, y tipo permite asignar una clase CSS distinta según la categoría del dato.

```
src > components > StatCard.jsx > default
1  function StatCard({ titulo, numero, tipo }) {
2    return (
3      <article className={`stat-card ${tipo}`}>
4        <p>{titulo}</p>
5        <strong>{numero}</strong>
6      </article>
7    );
8  }
9
10 export default StatCard;
```

Este componente recibe tres props: titulo, numero y tipo. La prop titulo indica el dato mostrado, numero representa el valor y tipo permite aplicar una clase CSS distinta según la categoría.

Su importancia está en que permite reutilizar una misma estructura para diferentes datos. Esto responde al uso de componentes reutilizables solicitado en la pauta del informe, donde se exige explicar componentes, props y lógica interna dentro del desarrollo en React

### Componente MenuNav

El componente MenuNav se utiliza para mostrar el menú de navegación principal de la aplicación. Este componente permite acceder a las distintas secciones del sistema, como Inicio, Contacto, Tickets, Acceso a la cuenta y Registro.

Su función es separar la navegación del resto de la interfaz, para que el menú no tenga que repetirse en cada página. Además, trabaja junto con React Router mediante enlaces internos, permitiendo cambiar de vista sin recargar completamente el sitio.

```
src > components > MenuNav.jsx > ...  
1 import { Link } from 'react-router-dom';  
2 import './MenuNav.scss';  
3  
4 export const MenuNav = ({ links = [] }) => {  
5   const defaultLinks = [  
6     { label: 'INICIO', path: '/' },  
7     { label: 'CONTACTO', path: '/contacto' },  
8     { label: 'TICKETS', path: '/tickets' },  
9     { label: 'ACCEDEA TU CUENTA', path: '/login' },  
10    { label: 'REGISTRATE', path: '/registro' }  
11  ];  
12
```

La lógica de este componente es simple: agrupa las rutas principales en una sola sección de navegación. También aporta a la estructura semántica del proyecto, ya que utiliza la etiqueta <nav>, correspondiente a navegación principal.

En el informe del equipo se indica que la navegación debe estar implementada con React Router y gestionada por un componente de menú, usando rutas y enlaces internos sin recargar el navegador.

### Componente Mensaje

El componente Mensaje se utiliza para mostrar retroalimentación al usuario cuando realiza una acción dentro del sistema. Por ejemplo, puede mostrar una confirmación al enviar un formulario o un aviso de error si falta completar algún campo.

### Componente MenuNav

El componente MenuNav se utiliza para mostrar la navegación principal del sitio. Permite acceder a secciones como Inicio, Contacto, Tickets, Accede a tu cuenta y Regístrate. Este componente evita repetir el menú en cada página y se relaciona con React Router, ya que sus enlaces permiten cambiar de vista sin recargar el navegador.



```

src > components > MenuNav.jsx > MenuNav
1  import { Link } from 'react-router-dom';
2  import './MenuNav.scss';
3
4  export const MenuNav = ({ links = [] }) => {
5    const defaultLinks = [
6      { label: 'INICIO', path: '/' },
7      { label: 'CONTACTO', path: '/contacto' },
8      { label: 'TICKETS', path: '/tickets' },
9      { label: 'ACCEDEA TU CUENTA', path: '/login' },
10     { label: 'REGISTRATE', path: '/registro' }
11   ];
12
13   const menuLinks = links.length > 0 ? links : defaultLinks;
14
15   return (
16     <nav className="menu-nav" role="navigation" aria-label="Navegación principal">
17       <ul className="menu-nav_list">
18         {menuLinks.map((link, index) => (
19           <li key={index} className="menu-nav_item">
20             <Link
21               to={link.path}
22               className="menu-nav_link"
23               aria-current={window.location.pathname === link.path ? 'page' : undefined}
24             >
25               {link.label}
26             </Link>
27           </li>
28         ))}
29       </ul>
30     </nav>
31   );
32 };
33
34 export default MenuNav;
35

```

## Componente TicketCard

El componente TicketCard se utiliza para mostrar información resumida de un ticket. Su función es presentar datos como título, estado, prioridad o descripción en una tarjeta reutilizable. Esto permite mostrar varios tickets usando la misma estructura, cambiando solo la información recibida por props.

```

src > components > TicketCard.jsx > ...
1  import './TicketCard.scss';
2
3  export const TicketCard = ({ nombre, codigo, estado, cantidad, imagen = 'img' }) => {
4    const estadoClass = estado.toLowerCase().replace(' ', '-');
5
6    return (
7      <div className="ticket-card" role="article">
8        <div className="ticket-card_header">
9          <span className="ticket-card_icon" aria-hidden="true">{imagen}</span>
10         <span className={`ticket-card_estado ticket-card_estado--${estadoClass}`}>
11           {estado}
12         </span>
13       </div>
14
15       <div className="ticket-card_content">
16         <h3 className="ticket-card_nombre">{nombre}</h3>
17         <p className="ticket-card_codigo">Código: {codigo}</p>
18
19         <div className="ticket-card_footer">
20           <span className="ticket-card_cantidad">
21             <strong>{cantidad}</strong> eventos
22           </span>
23         </div>
24       </div>
25     </div>
26   );
27 };
28
29 export default TicketCard;
30

```

## Hooks utilizados: useState

El hook useState se utilizó para manejar cambios dentro de la interfaz. En la pantalla de ingreso permite controlar si se muestra la vista inicial o el panel de

usuario. También permite cambiar entre secciones internas, como Nuevo ticket, Registro de comunicaciones y Ayuda.

```
src > pages > Login.jsx > Login
1  import { useState } from "react";
2  import HomeCard from "../components/HomeCard";
3  import "../Login.scss";
4
5  function Login() {
6    const [vista, setVista] = useState("inicio");
7    const [seccion, setSeccion] = useState("nuevo");
8  }
```

Cuando el usuario presiona “USUARIO”, se actualiza el estado y se muestra el formulario de ticket sin recargar la página.

### Hooks utilizados: useEffect

El hook useEffect se utiliza para ejecutar acciones después del renderizado del componente. En el componente Mensaje, se usa para cerrar automáticamente el aviso después de unos segundos. Esto permite mejorar la interacción con el usuario sin necesidad de recargar la página.

```
src > components > Mensaje.jsx > ...
1  import './Mensaje.scss';
2  import { useEffect } from 'react';
3
4  export const Mensaje = ({ tipo = 'exito', texto = '', visible = false, onDismiss = () => {} }) => {
5    useEffect(() => {
6      if (visible) {
7        const timer = setTimeout(() => {
8          onDismiss();
9        }, 5000);
10       return () => clearTimeout(timer);
11     }
12   })
13 }
```

### React Router

React Router se utilizó para definir las rutas principales de la aplicación. Esto permite navegar entre páginas como Inicio, Contacto, Tickets, Login y Registro sin recargar completamente el sitio. La configuración se realiza en App.jsx, donde cada ruta se asocia a un componente.

| Ruta      | Componente | Función                          |
|-----------|------------|----------------------------------|
| /         | Home       | Página Principal                 |
| /contacto | Contacto   | Formulario o sección de contacto |
| /ticket   | Tickets    | Vista de tickets                 |
| /login    | Login      | Acceso al sistema                |
| /registro | Registro   | Registro de usuario              |

### **Flujo de dato entre componentes**

El flujo de datos se realiza principalmente mediante props y estado local. Los componentes principales, como Login.jsx, manejan estados con useState y envían información a componentes reutilizables como HomeCard, Mensaje o TicketCard. De esta forma, los componentes hijos reciben datos y funciones desde el componente padre.

### **Cierre de la seccion**

En conclusión, React permitió organizar la aplicación en páginas, componentes reutilizables, rutas y estados. Esta estructura facilita el mantenimiento del código y permite explicar con claridad cómo se construye la interfaz. El uso de props, hooks y React Router ayuda a que la aplicación sea más ordenada, funcional y coherente con los requerimientos del sistema de tickets.

## 4. Conclusiones

El desarrollo de este sistema de gestión de tickets de mantenimiento demostró que la combinación de herramientas modernas permite crear aplicaciones profesionales, limpias y eficientes. Los resultados clave del proyecto son:

-Modularidad con React: La división de la interfaz en componentes reutilizables (TicketCard, HomeCard, StatCard) evitó duplicar código, haciendo que el sistema sea fácil de mantener y escalar en el futuro.

-Interactividad y Dinamismo: Gracias a React Router, la navegación entre las páginas (Inicio, Tickets, Contacto y Login) es instantánea y sin recargar el navegador. Además, el uso de useState y useEffect logró que la aplicación responda de inmediato a las acciones de los usuarios en la tienda.

-Estructura limpia y Accesible: El uso de HTML5 Semántico (<header>, <nav>, <main>, <article>, <footer>) junto con formularios correctamente vinculados (htmlFor/id), garantizó una base ordenada que cumple estrictamente con los estándares de accesibilidad web.

-Diseño Adaptable (Responsive): La arquitectura modular de SASS/SCSS (\_variables.scss y \_mixins.scss) facilitó el control de la identidad visual de la empresa. Al combinar CSS Grid y Flexbox, la interfaz se adapta automáticamente a computadores, tablets y celulares sin perder orden ni usabilidad.

En resumen, el proyecto cumplió con todos los requerimientos técnicos exigidos, entregando una interfaz sólida, accesible y lista para operar en un entorno real.